INF563 · VÉRIFICATION ET VALIDATION DE LOGICIEL · MASTER 2 SIGL

Devoir 1 – BDD en Java : connexion à mon compte GitHub

Scénarios Gherkin écrits en français, exécutés par **Cucumber** et **Playwright** contre le vrai site github.com ; code de double authentification développé en **TDD** ; tests déclenchés automatiquement à chaque **push** par GitHub Actions.

6 / 6	23 / 23	30	Vert
scénarios BDD exécutés réussis (+1 ignoré sans identifiants)	tests unitaires écrits en TDD	tests exécutés par *mvn verify*, aucun échec	pipeline GitHub Actions au 1er push (3 min 34 s)

Dépôt du devoir

github.com/dradone237/inf563-bdd-connexion-github · branche `main` · 6 commits · rapport Cucumber publié par la CI à chaque exécution.

Enseignant : Evaris Fomekong · Séance de référence : séance 9, « Niveaux, doublures et TDD/BDD », complétée par la séance 12 (« Bout en bout web : Playwright et page objects »)

Java 21 · Maven · JUnit 5.13.4 · Cucumber 7.34.9 · Playwright 1.64.0 · GitHub Actions

Rapport de devoir · 6 novembre 2026

Table des matières

1. Énoncé et compréhension du devoir **3**

1.1 Consignes 3

1.2 Ce que le devoir doit démontrer 3

1.3 Choix du site : GitHub plutôt que Facebook 3

2. Démarche BDD **3**

2.1 De la user story au scénario 3

2.2 Double boucle BDD / TDD 4

3. Architecture du projet **4**

3.1 Arborescence 4

3.2 Vue d'ensemble 4

3.3 Rôle de chaque classe 5

3.4 Bibliothèques 5

4. Scénarios et oracles **5**

4.1 Les sept scénarios 5

4.2 Choix des oracles 6

5. Implémentation des étapes **7**

5.1 Le page object 7

5.2 Les définitions d'étapes 7

5.3 Isolation et diagnostic 7

5.4 Le scénario sur le compte réel 8

6. Partie TDD : le générateur de codes de double authentification **8**

6.1 Pourquoi ce composant 8

6.2 Cycle rouge → vert 8

6.3 Jeux de test 8

6.4 Maîtriser le temps : l'horloge injectée 9

7. Intégration continue : le push déclenche les tests **9**

7.1 Le pipeline 9

7.2 Premier passage 9

8. Résultats **10**

8.1 Exécution locale 10

8.2 Historique Git 11

8.3 Défaut d'oracle corrigé pendant le développement 11

9. Lacunes de spécification et décisions **12**

10. Limites et pistes d'amélioration **12**

11. Conclusion **12**

Annexe A – Fichier connexion_github.feature **13**

Annexe B – Pipeline .github/workflows/ci.yml **13**

Annexe C – Générateur TOTP (Totp.java) **14**

Annexe D – Commandes utiles **16**

1. Énoncé et compréhension du devoir

1.1 Consignes

Implémenter **en Java** un test **BDD** (*Behaviour-Driven Development*) qui vérifie la **connexion à mon compte GitHub ou Facebook**. Les tests doivent être exécutés automatiquement **à chaque push** sur le dépôt.

Le devoir porte sur la séance 9 du cours, « Niveaux, doublures et TDD/BDD », qui présente le BDD comme une *spécification exécutable* écrite en Gherkin et reliée au code par Cucumber, puis son exécution dans un pipeline GitHub Actions. L'outil de bout en bout (Playwright, page objects) vient de la séance 12.

1.2 Ce que le devoir doit démontrer

- écrire le comportement attendu **en langage métier** (Gherkin, en français) avant le code qui le vérifie ;
- relier chaque phrase à du **code Java exécutable** (définitions d'étapes Cucumber) ;
- piloter un **vrai navigateur** sur un **vrai site** : c'est un test de bout en bout, au niveau *système/acceptation* de la pyramide des tests ;
- automatiser l'exécution dans un **pipeline d'intégration continue** déclenché par le push.

1.3 Choix du site : GitHub plutôt que Facebook

Facebook détecte et bloque activement l'automatisation : « checkpoint » de sécurité, captcha, blocage temporaire du compte. Un test qui échoue pour une raison extérieure au comportement testé est un **test instable** (*flaky*), ce que le cours demande d'éliminer. GitHub offre au contraire :

- un formulaire stable, aux identifiants HTML fixes (`#login_field`, `#password`) ;
- des messages d'erreur constants (« Incorrect username or password. ») ;
- l'utilisateur connecté publié dans la balise `<meta name="user-login">` (vide si personne n'est connecté) : un **oracle** fiable de l'état de connexion, indépendant de la mise en page.

2. Démarche BDD

2.1 De la user story au scénario

La fonctionnalité est d'abord exprimée comme une user story (« Afin de… En tant que… Je veux… »), puis déclinée en scénarios concrets. Chaque scénario suit la structure **Soit / Quand / Alors**, l'équivalent Gherkin de **Arrange / Act / Assert** vu en cours pour les tests unitaires.

```
# language: fr
@github
Fonctionnalité: Connexion à mon compte GitHub
  Afin d'accéder à mes dépôts et de pousser mon code
  En tant que propriétaire d'un compte GitHub
  Je veux me connecter avec mon identifiant et mon mot de passe
  et que toute tentative avec de mauvais identifiants soit refusée

  Contexte:
    Soit je suis sur la page de connexion de GitHub
```

La ligne `# language: fr` active les mots-clés français. Le **Contexte** est rejoué avant chaque scénario : chacun part de la page de connexion, dans un navigateur vierge.

2.2 Double boucle BDD / TDD

Le cours décrit une **double boucle** : un scénario d'acceptation rouge (boucle externe, BDD) est rendu vert par une série de cycles TDD (boucle interne). Ce devoir l'applique : le scénario de connexion au compte réel a besoin, si la double authentification est activée, d'un générateur de codes à usage unique. Ce composant a été développé en TDD (section 6) avant d'être utilisé par le page object.

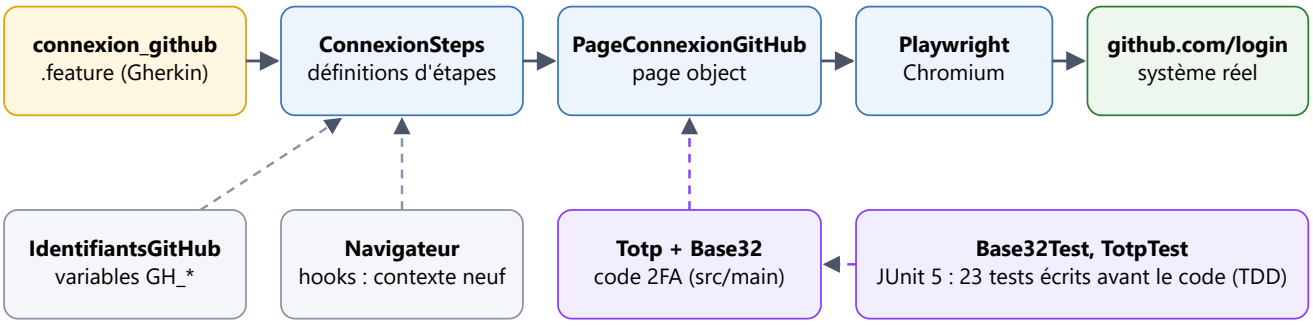
Boucle	Qui / niveau	Langage	Dans ce devoir
BDD (externe)	métier + dév + test ; comportement, acceptation	Gherkin	7 scénarios sur la page de connexion GitHub
TDD (interne)	développeur ; unitaire	Java (JUnit 5)	23 tests de `Base32` et `Totp`

3. Architecture du projet

3.1 Arborescence

```
inf563-bdd-connexion-github/
├── .github/workflows/ci.yml                 pipeline : push → tests
├── pom.xml                                  Java 21, JUnit 5, Cucumber, Playwright
└── src/
    ├── main/java/cm/inf563/github/securite/
    │   ├── Base32.java                      décodage de la clé secrète 2FA
    │   └── Totp.java                        code à 6 chiffres (RFC 6238)
    └── test/
        ├── java/cm/inf563/github/
        │   ├── RunCucumberTest.java         point d'entrée JUnit 5 de Cucumber
        │   ├── pages/PageConnexionGitHub.java   page object
        │   ├── steps/ConnexionSteps.java        phrases → actions et assertions
        │   ├── support/Navigateur.java          cycle de vie de Chromium (hooks)
        │   ├── support/IdentifiantsGitHub.java  variables d'environnement
        │   └── securite/Base32Test.java, TotpTest.java   tests TDD
        └── resources/features/connexion_github.feature
```

3.2 Vue d'ensemble



Flèches pleines : chemin d'exécution d'un scénario · pointillés : composants de support · violet : partie TDD

Figure 1 – Architecture : chaque phrase Gherkin est reliée à une étape Java, qui agit sur la page réelle de GitHub au travers d'un page object et de Playwright.

3.3 Rôle de chaque classe

Classe	Rôle
`RunCucumberTest`	Suite JUnit 5 (`@Suite` , `@IncludeEngines("cucumber")`) : exécute toutes les features et produit les rapports HTML et JUnit XML.
`ConnexionSteps`	Définitions d'étapes : une méthode annotée (`@Etantdonné` , `@Quand` , `@Alors`) par phrase type ; paramètres extraits par les expressions `{string}` .
`PageConnexionGitHub`	Page object : seul endroit où figurent les sélecteurs ; expose des intentions (`saisir` , `valider` , `seConnecter` , `seDeconnecter`).
`Navigateur`	Hooks Cucumber : un Chromium pour toute la suite, un contexte neuf par scénario, capture d'écran jointe au rapport en cas d'échec.
`IdentifiantsGitHub`	Lit `GH_LOGIN` , `GH_PASSWORD` , `GH_TOTP_SECRET` ; masque le mot de passe dans `toString()` .
`Base32` , `Totp`	Code de production développé en TDD : calcul du code de double authentification.

3.4 Bibliothèques

Bibliothèque	Version	Usage
Java (Temurin)	21	langage, `Clock` , `javax.crypto.Mac` (HMAC-SHA1)
JUnit 5 (Jupiter + Platform Suite)	5.13.4	tests unitaires, tests paramétrés, lancement de Cucumber
Cucumber JVM (java, junit-platform-engine, picocontainer)	7.34.9	exécution du Gherkin, injection de dépendances entre hooks et étapes
Playwright pour Java	1.64.0	pilotage de Chromium, assertions qui attendent automatiquement
Maven Surefire	3.5.4	exécution des tests par `mvn verify`

4. Scénarios et oracles

4.1 Les sept scénarios

#	Scénario	Type	Oracle (ce qui est vérifié)
1	La page de connexion présente le formulaire complet	nominal	champ identifiant éditable, champ mot de passe de type `password` , bouton « Sign in » actif, personne n'est connecté
2	Un formulaire vide n'est pas envoyé	valeur limite	`validity.valueMissing` sur l'identifiant ; l'URL reste `/login`
3	Un mot de passe vide n'est pas envoyé	valeur limite	`validity.valueMissing` sur le mot de passe ; l'URL reste `/login`
4	Plan : identifiants invalides – compte inexistant	erreur	message « Incorrect username or password. » ; toujours non connecté
5	Plan : identifiants invalides – adresse e-mail inconnue	erreur	même message ; toujours non connecté

6	Le lien « mot de passe oublié » mène à la réinitialisation	navigation	URL `github.com/password_reset`
7	`@compte-reel` Connexion réussie à mon compte puis déconnexion	nominal (parcours critique)	`meta user-login` = mon identifiant, puis vide après la déconnexion

Les scénarios 4 et 5 forment un **plan du scénario** (*Scenario Outline*) : une seule description, un tableau d'exemples, un cas de test par ligne, comme `@ParameterizedTest` pour JUnit. Les identifiants invalides sont fictifs : aucun vrai compte n'est sollicité, ce qui évite de déclencher les protections de GitHub.

4.2 Choix des oracles

- **État de connexion** : la balise `meta[name=user-login]`, renseignée par GitHub sur toutes ses pages. Elle ne dépend ni de la langue ni de la mise en page.
- **Champs obligatoires** : l'API de validation HTML5 (`validity.valueMissing`). Elle prouve que c'est le navigateur qui bloque l'envoi, sans requête vers GitHub.
- **Message d'erreur** : le bandeau `.flash-error`. La page en contient un second, caché (`#ajax-error-message`), que le sélecteur exclut (voir 8.3).
- **Navigation** : l'URL de la page, vérifiée par `assertThat(page).hasURL(...)`.

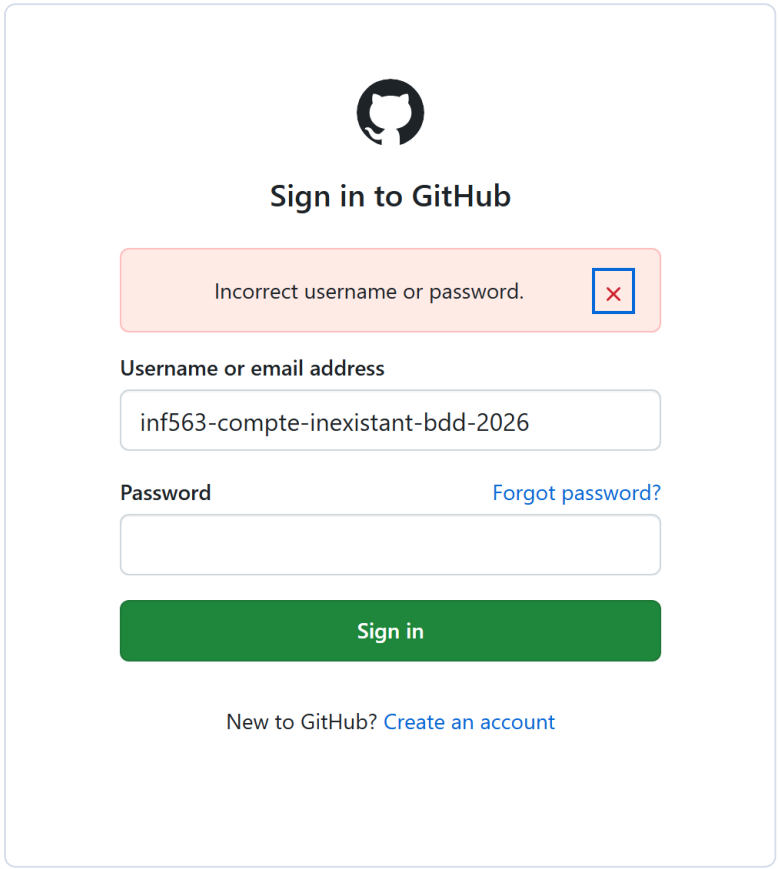


Figure 2 – État vérifié par le plan du scénario « identifiants invalides » : le message d'erreur est affiché et personne n'est connecté (capture réalisée par Playwright).

5. Implémentation des étapes

5.1 Le page object

Les sélecteurs n'apparaissent que dans `PageConnexionGitHub` ; ils reposent sur des identifiants HTML et des rôles ARIA, jamais sur des XPath fragiles. Si GitHub modifie sa page, une seule classe change.

```
public Locator messageErreur() {
    // La page contient aussi un bandeau .flash-error caché (#ajax-error-message) : on l'exclut
    return page.locator(".flash-error:not(#ajax-error-message)");
}

/** GitHub publie l'utilisateur connecté dans <meta name="user-login"> (vide si anonyme). */
public Locator utilisateurConnecte() {
    return page.locator("meta[name='user-login']");
}

public void seConnecter(IdentifiantsGitHub identifiants, Clock horloge) {
    saisir(identifiants.identifiant(), identifiants.motDePasse());
    valider();
    page.waitForURL(url -> !url.endsWith("/login"));
    if (page.url().contains("/sessions/verified-device")) {
        throw new IllegalStateException("GitHub demande un code de vérification envoyé par e-mail. ...");
    }
    if (page.url().contains("/sessions/two-factor")) {
        saisirCodeDoubleAuthentification(identifiants, horloge);
    }
}
```

5.2 Les définitions d'étapes

Chaque phrase Gherkin correspond à une méthode. Les assertions Playwright (`assertThat(locator)`) **attendent automatiquement** que la condition soit vraie, dans la limite du délai configuré : il n'y a aucun `Thread.sleep`, cité par le cours comme anti-pattern.

```
@Quand("je saisis l'identifiant {string} et le mot de passe {string}")
public void jeSaisis(String identifiant, String motDePasse) {
    connexion.saisir(identifiant, motDePasse);
}

@Alors("le message d'erreur {string} s'affiche")
public void leMessageDErreurSAffiche(String message) {
    assertThat(connexion.messageErreur()).containsText(message);
}

@Alors("je ne suis pas connecté")
public void jeNeSuisPasConnecte() {
    assertThat(connexion.utilisateurConnecte()).hasAttribute("content", "");
}
```

5.3 Isolation et diagnostic

- **Un seul Chromium** démarré pour toute la suite (`@BeforeAll`), car son lancement est lent ;
- **un contexte de navigation neuf par scénario** (`@Before`) : cookies et session vides. Les scénarios sont *isolés* et *reproductibles* (F.I.R.S.T.) et peuvent s'exécuter dans n'importe quel ordre ;
- **langue forcée à** `en-US` : les textes vérifiés ne dépendent pas de la configuration du poste ;
- **capture d'écran pleine page jointe au rapport Cucumber** en cas d'échec (`@After`) ;

- options de démonstration : `-Dheadless=false` (navigateur visible) et `-DslowMo=400` (actions ralenties).

5.4 Le scénario sur le compte réel

Les identifiants ne sont **jamais écrits dans le code** : ils sont lus dans des variables d'environnement, qui sont des *secrets* du dépôt en CI. Le préfixe `GITHUB_` étant réservé par GitHub Actions, les variables commencent par `GH_` .

Variable	Contenu	Obligatoire
`GH_LOGIN`	identifiant ou adresse e-mail du compte	oui
`GH_PASSWORD`	mot de passe	oui
`GH_TOTP_SECRET`	clé Base32 de l'application d'authentification (« setup key »)	si la 2FA est activée

En l'absence de `GH_LOGIN` ou `GH_PASSWORD`, un hook `@Before("@compte-reel")` appelle `Assumptions.assumeTrue(...)` : le scénario est marqué **ignoré** et non en échec. Le pipeline reste ainsi utilisable par tout le monde, y compris pour une *pull request* venant d'un autre dépôt, qui n'a pas accès aux secrets.

6. Partie TDD : le générateur de codes de double authentification

6.1 Pourquoi ce composant

GitHub exige la double authentification sur la plupart des comptes. Pour que le scénario sur le compte réel s'exécute sans intervention humaine, le projet calcule lui-même le code à 6 chiffres affiché d'habitude par l'application d'authentification : c'est l'algorithme **TOTP** (RFC 6238). Il applique HMAC-SHA1 au numéro de la fenêtre de 30 s, puis une troncature dynamique (RFC 4226) ; la clé secrète est fournie en **Base32** (RFC 4648).

6.2 Cycle rouge → vert

Les tests ont été écrits **avant** le code, comme le montre l'historique Git du dépôt :

Commit	Étape	État
`83d04f3`	tests `Base32Test` et `TotpTest` tirés des vecteurs officiels	**rouge** : ne compile pas (« ne pas compiler compte comme un échec », loi 2 du TDD)
`eb1ee84`	implémentation de `Base32` et `Totp`	**vert** : 23 tests réussis

6.3 Jeux de test

L'**oracle** est fourni par les spécifications elles-mêmes : les vecteurs de test officiels des RFC. Les autres cas viennent de l'analyse en classes d'équivalence et valeurs limites (séance 4).

Classe / technique	Données	Résultat attendu
Vecteurs RFC 4648 (Base32)	`MY======` … `MZXW6YTBOI======` (6 cas)	`f` , `fo` , … `foobar`
Vecteurs RFC 6238, 8 chiffres	t = 59, 1111111109, 1111111111, 1234567890, 2000000000, 20000000000 s	`94287082` , `07081804` , `14050471` , `89005924` , `69279037` , `65353130`

6 chiffres (format GitHub), zéros non significatifs	t = 59, 1111111109, 1234567890 s	`287082` , `081804` , `005924`
Valeurs limites de la fenêtre de 30 s	t = 30, 59, 60 s	30 et 59 donnent le même code, 60 donne le suivant
Format saisi par un humain	`mzxw 6ytb-oi`	identique à `MZXW6YTBOI`
Classes invalides	caractère `1` , secret vide, `null` , 5 ou 9 chiffres	exception explicite

6.4 Maîtriser le temps : l'horloge injectée

Comme dans le cours (« maîtriser le temps : injecter une Clock »), `Totp` reçoit une `java.time.Clock` au lieu d'appeler l'heure système. Les tests utilisent `Clock.fixed(...)` et donnent toujours le même résultat ; le scénario BDD utilise `Clock.systemUTC()` .

```
private static Clock horlogeA(long secondesEpoch) {
    return Clock.fixed(Instant.ofEpochSecond(secondesEpoch), ZoneOffset.UTC);
}

@Test
@DisplayName("Bornes d'une fenêtre de 30 s : 30 et 59 → même code, 60 → code suivant")
void fenetreDeTrenteSecondes() {
    String debut = new Totp(SECRET_RFC, horlogeA(30)).codeCourant();
    String fin = new Totp(SECRET_RFC, horlogeA(59)).codeCourant();
    String suivant = new Totp(SECRET_RFC, horlogeA(60)).codeCourant();
    assertEquals(debut, fin);
    assertNotEquals(fin, suivant);
}
```

7. Intégration continue : le push déclenche les tests

7.1 Le pipeline

Le fichier `.github/workflows/ci.yml` (annexe B) se déclenche à chaque `push` sur n'importe quelle branche, à chaque *pull request* et à la demande (`workflow_dispatch`). Il enchaîne :

1. `actions/checkout` puis `actions/setup-java` : JDK Temurin 21 et cache des dépendances Maven ;
2. installation de Chromium et de ses dépendances système par le CLI Playwright, **de la même version** que la bibliothèque Java (sinon le navigateur ne correspond pas au pilote) ;
3. `mvn -B verify` : tests unitaires TDD puis scénarios Cucumber, avec les secrets en variables d'environnement ;
4. publication des rapports Cucumber (HTML, captures d'échec incluses) et Surefire, **même en cas d'échec** (`if: always()`).

7.2 Premier passage

Le premier push sur `main` (commit `7e2858c`) a déclenché l'exécution n° 1 : **succès** en 3 min 34 s, avec un artefact `rapports-tests` publié. Les scénarios contre github.com ont donc réussi depuis les serveurs de GitHub Actions comme depuis le poste de développement.

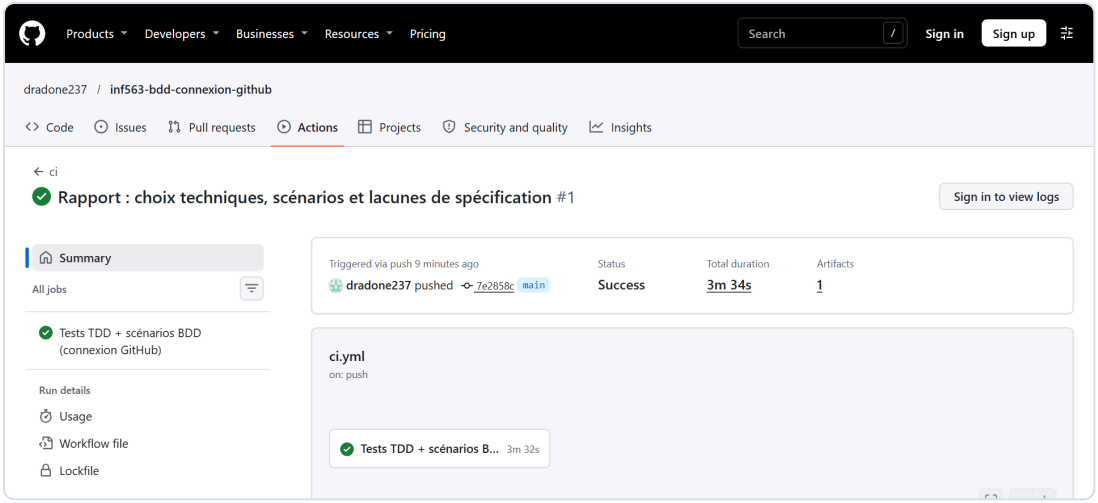


Figure 3 – Exécution n° 1 du pipeline, déclenchée par le push : statut « Success » (github.com/dradone237/inf563-bdd-connexion-github/actions/runs/42291225691).

> Règle recommandée par le cours : protéger la branche `main` (« Require status checks to pass ») pour qu'un pipeline rouge interdise la fusion.

8. Résultats

8.1 Exécution locale

Suite	Tests	Réussis	Ignorés	Échecs	Durée
Scénarios Cucumber (`RunCucumberTest`)	7	6	1	0	≈ 25 s
Décodage Base32 (`Base32Test`)	10	10	0	0	0,1 s
Codes TOTP (`TotpTest`)	13	13	0	0	0,05 s
Total `mvn verify`	**30**	**29**	**1**	**0**	**BUILD SUCCESS**

Le scénario ignoré est `@compte-reel` : il attend les identifiants du compte (section 5.4).

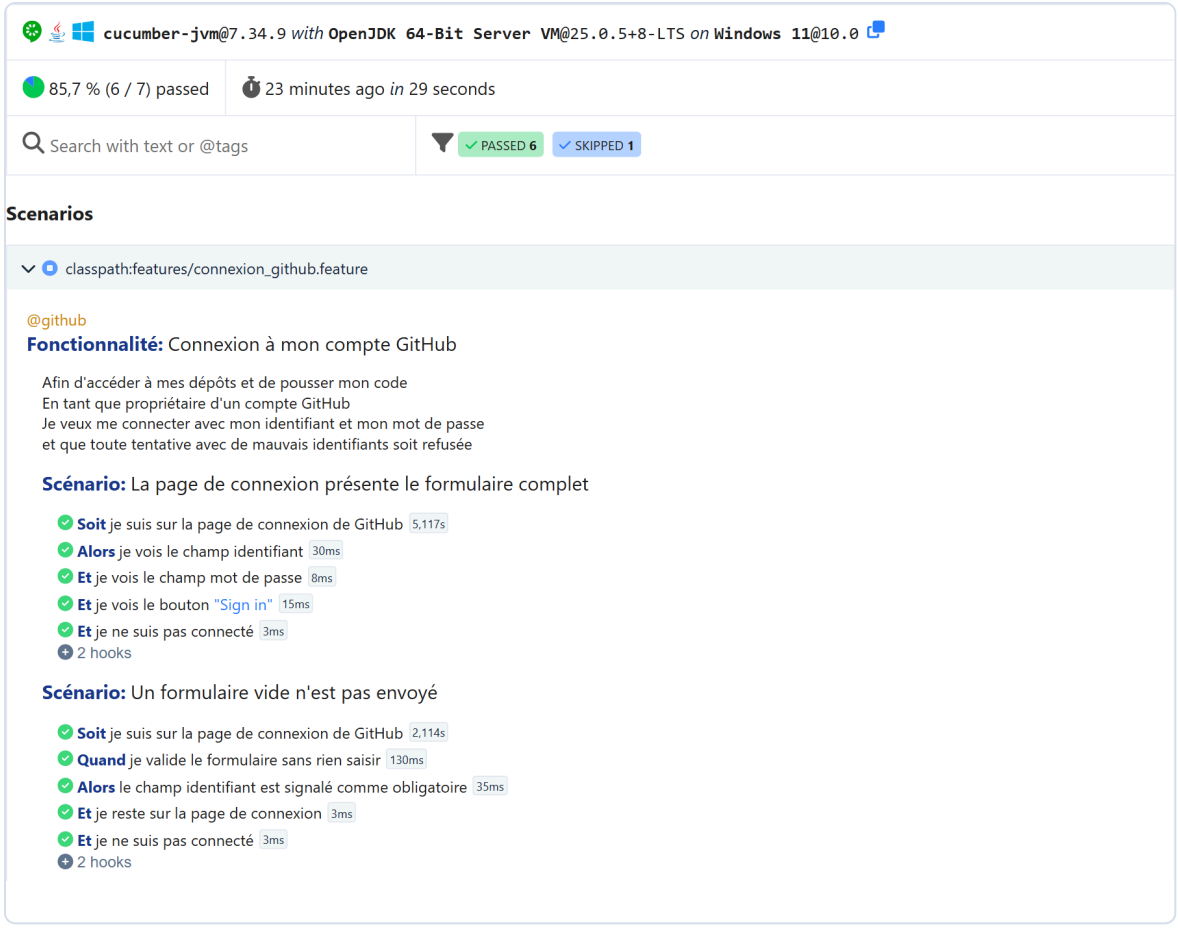


Figure 4 – Rapport HTML produit par Cucumber : 6 scénarios réussis, 1 ignoré ; chaque étape affiche sa durée.

8.2 Historique Git

Commit	Message
`4c2a46d`	Projet Maven : Java 21, JUnit 5, Cucumber et Playwright
`83d04f3`	TDD (rouge) : tests Base32 et TOTP tirés des vecteurs des RFC 4648 et 6238
`eb1ee84`	TDD (vert) : décodeur Base32 et générateur de codes TOTP à horloge injectée
`dcbf8d6`	BDD : scénarios Gherkin de connexion à GitHub, page object et étapes Playwright
`ccdd3a9`	CI : GitHub Actions lance les tests à chaque push
`7e2858c`	Rapport : choix techniques, scénarios et lacunes de spécification

8.3 Défaut d'oracle corrigé pendant le développement

À la première exécution, les deux exemples du plan « identifiants invalides » ont échoué avec `strict mode violation: locator(".flash-error") resolved to 2 elements`. GitHub insère dans toutes ses pages un second bandeau d'erreur, **caché** (`#ajax-error-message`), réservé aux erreurs de requêtes asynchrones. Le mode strict de Playwright, qui refuse un sélecteur ambigu, a révélé que l'oracle n'était pas assez précis. Le sélecteur exclut désormais ce bandeau. Ce n'était pas un défaut de GitHub mais du test : **un test aussi doit être vérifié**.

9. Lacunes de spécification et décisions

Question non tranchée par l'énoncé	Décision
GitHub ou Facebook ?	GitHub : formulaire stable, oracle fiable, pas de blocage anti-robot systématique (section 1.3).
Les textes de GitHub dépendent de la langue du navigateur.	Langue forcée à `en-US` dans le contexte de navigation.
La connexion réelle peut exiger la 2FA ou une vérification par e-mail.	2FA par application gérée (TOTP, section 6) ; vérification par e-mail détectée avec un message explicite.
`GH_LOGIN` peut être une adresse e-mail, alors que GitHub affiche le nom d'utilisateur.	Avec une adresse e-mail, on vérifie seulement qu'un utilisateur est connecté.
Que faire sans identifiants (poste d'un camarade, pull request externe) ?	Scénario `@compte-reel` ignoré, les autres s'exécutent.

10. Limites et pistes d'amélioration

- Les scénarios dépendent du **vrai site GitHub** : réseau, évolution de la page, limitation après des échecs répétés depuis une même adresse IP. C'est le prix d'un vrai test de bout en bout ; ils restent peu nombreux (pyramide des tests) et chaque échec est accompagné d'une capture d'écran.
- Pour le scénario sur le compte réel en CI, un **compte GitHub de test dédié** est préférable au compte principal : moins de risque d'alerte de sécurité ou de blocage.
- Le bouton de déconnexion n'a pas pu être vérifié sans identifiants ; il sera exercé dès que les secrets seront configurés.
- Piste : exécuter aussi les scénarios dans Firefox et WebKit (Playwright les prend en charge) via une matrice GitHub Actions.

11. Conclusion

Le comportement de la connexion à GitHub est décrit en français dans un fichier Gherkin lisible par un non-développeur, et chaque phrase est exécutée par du code Java contre le vrai site. Les choix suivent les recommandations du cours : page object, sélecteurs stables, assertions sans attente fixe, contexte isolé par scénario, captures en cas d'échec, identifiants hors du code. Le seul composant algorithmique, le code de double authentification, a été développé en TDD à partir des vecteurs officiels des RFC, avec une horloge injectée. Enfin, chaque push déclenche le pipeline GitHub Actions, vert dès le premier envoi.

Annexe A – Fichier connexion_github.feature

```
# language: fr
@github
Fonctionnalité: Connexion à mon compte GitHub
  Afin d'accéder à mes dépôts et de pousser mon code
  En tant que propriétaire d'un compte GitHub
  Je veux me connecter avec mon identifiant et mon mot de passe
  et que toute tentative avec de mauvais identifiants soit refusée

  Contexte:
    Soit je suis sur la page de connexion de GitHub

  Scénario: La page de connexion présente le formulaire complet
    Alors je vois le champ identifiant
    Et je vois le champ mot de passe
    Et je vois le bouton "Sign in"
    Et je ne suis pas connecté

  Scénario: Un formulaire vide n'est pas envoyé
    Quand je valide le formulaire sans rien saisir
    Alors le champ identifiant est signalé comme obligatoire
    Et je reste sur la page de connexion
    Et je ne suis pas connecté

  Scénario: Un mot de passe vide n'est pas envoyé
    Quand je saisis l'identifiant "inf563-etudiant" et le mot de passe ""
    Et je valide le formulaire
    Alors le champ mot de passe est signalé comme obligatoire
    Et je reste sur la page de connexion

  Plan du scénario: Connexion refusée avec des identifiants invalides (<cas>)
    Quand je saisis l'identifiant "<identifiant>" et le mot de passe "<mot de passe>"
    Et je valide le formulaire
    Alors le message d'erreur "Incorrect username or password." s'affiche
    Et je ne suis pas connecté

    Exemples:
      | cas                    | identifiant                       | mot de passe    |
      | compte inexistant      | inf563-compte-inexistant-bdd-2026 | MotDePasse!2026 |
      | adresse e-mail inconnue | inf563.inconnu@example.invalid    | MotDePasse!2026 |

  Scénario: Le lien « mot de passe oublié » mène à la réinitialisation
    Quand je clique sur le lien "Forgot password?"
    Alors je suis sur la page de réinitialisation du mot de passe

  # Ce scénario utilise MON vrai compte : identifiants lus dans les variables
  # d'environnement GH_LOGIN, GH_PASSWORD et (si 2FA activée) GH_TOTP_SECRET.
  # Sans ces variables, il est marqué « ignoré » (skipped) et non en échec.
  @compte-reel
  Scénario: Connexion réussie à mon compte puis déconnexion
    Quand je me connecte avec mes identifiants personnels
    Alors je suis connecté à mon compte GitHub
    Quand je me déconnecte
    Alors je ne suis pas connecté
```

Annexe B – Pipeline .github/workflows/ci.yml

```
# Intégration continue : chaque push (sur n'importe quelle branche) et chaque
# pull request déclenche automatiquement les tests TDD puis les scénarios BDD.
# Un pipeline rouge = on ne fusionne pas (règle de protection de la branche main).
```

```yaml
name: ci

on:
  push:
  pull_request:
  workflow_dispatch:   # bouton « Run workflow » pour relancer à la main

jobs:
  tests:
    name: "Tests TDD + scénarios BDD (connexion GitHub)"
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: maven

      - name: Installer Chromium et ses dépendances système (Playwright)
        run: >
          mvn -B -q exec:java -e
          -Dexec.mainClass=com.microsoft.playwright.CLI
          -Dexec.args="install --with-deps chromium"
          -Dexec.classpathScope=test

      - name: Tests unitaires (TDD) puis scénarios Gherkin (BDD)
        run: mvn -B verify
        env:
          PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD: '1'
          # Secrets du dépôt (Settings > Secrets and variables > Actions).
          # Absents : le scénario @compte-reel est ignoré, les autres s'exécutent.
          GH_LOGIN: ${{ secrets.GH_LOGIN }}
          GH_PASSWORD: ${{ secrets.GH_PASSWORD }}
          GH_TOTP_SECRET: ${{ secrets.GH_TOTP_SECRET }}

      - name: Publier les rapports (même en cas d'échec)
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: rapports-tests
          path: |
            target/cucumber-reports/
            target/surefire-reports/
```

Annexe C – Générateur TOTP (Totp.java)

```java
package cm.inf563.github.securite;

import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.ByteBuffer;
import java.security.GeneralSecurityException;
import java.time.Clock;
import java.util.Objects;

/**
 * Générateur de codes à usage unique basés sur le temps (TOTP, RFC 6238),
 * identique à Google Authenticator : HMAC-SHA1, pas de 30 secondes.
 * <p>
 * L'horloge est injectée (cf. cours, séance 9 : « maîtriser le temps ») :
```

```java
 * les tests utilisent {@code Clock.fixed(...)}, la production {@code Clock.systemUTC()}.
 */
public final class Totp {

    private static final long PAS_SECONDES = 30;
    private static final String ALGORITHME = "HmacSHA1";

    private final byte[] secret;
    private final Clock horloge;
    private final int chiffres;
    private final int modulo;

    public Totp(byte[] secret, Clock horloge) {
        this(secret, horloge, 6);
    }

    public Totp(byte[] secret, Clock horloge, int chiffres) {
        Objects.requireNonNull(secret, "secret");
        if (secret.length == 0) {
            throw new IllegalArgumentException("Le secret TOTP est vide");
        }
        if (chiffres < 6 || chiffres > 8) {
            throw new IllegalArgumentException("Un code TOTP a entre 6 et 8 chiffres, pas " + chiffres);
        }
        this.secret = secret.clone();
        this.horloge = Objects.requireNonNull(horloge, "horloge");
        this.chiffres = chiffres;
        this.modulo = (int) Math.pow(10, chiffres);
    }

    public static Totp depuisSecretBase32(String secretBase32, Clock horloge) {
        return new Totp(Base32.decoder(secretBase32), horloge);
    }

    public String codeCourant() {
        long compteur = horloge.instant().getEpochSecond() / PAS_SECONDES;
        byte[] empreinte = hmac(ByteBuffer.allocate(Long.BYTES).putLong(compteur).array());

        // Troncature dynamique (RFC 4226, section 5.3)
        int decalage = empreinte[empreinte.length - 1] & 0x0F;
        int binaire = ((empreinte[decalage] & 0x7F) << 24)
                | ((empreinte[decalage + 1] & 0xFF) << 16)
                | ((empreinte[decalage + 2] & 0xFF) << 8)
                | (empreinte[decalage + 3] & 0xFF);

        return String.format("%0" + chiffres + "d", binaire % modulo);
    }

    private byte[] hmac(byte[] message) {
        try {
            Mac mac = Mac.getInstance(ALGORITHME);
            mac.init(new SecretKeySpec(secret, ALGORITHME));
            return mac.doFinal(message);
        } catch (GeneralSecurityException e) {
            throw new IllegalStateException("HMAC-SHA1 indisponible dans cette JVM", e);
        }
    }
}
```

Annexe D – Commandes utiles

```
# Une seule fois : télécharger Chromium pour Playwright
mvn exec:java -e -Dexec.mainClass=com.microsoft.playwright.CLI -Dexec.args="install chromium" -
Dexec.classpathScope=test

mvn verify                                  # tests TDD + scénarios BDD
mvn verify -Dheadless=false -DslowMo=400    # démonstration : navigateur visible et ralenti

# Avec mon compte (PowerShell)
$env:GH_LOGIN="mon-identifiant"; $env:GH_PASSWORD="..."; $env:GH_TOTP_SECRET="..."; mvn verify

# Rapport lisible : target/cucumber-reports/cucumber.html
```